BURHAN KOCABIYIK · AÇIK KAYNAK VİDEO MODELİ

2.3

ltx · video + ses · tek model

Video modelini *kendi makinende çalıştır.*

LTX-2.3, Lightricks'in açık ağırlıklı video modeli. Videoyu ve **senkron sesi tek geçişte** üretiyor; yazıdan videoya, görselden videoya, sesten videoya çalışıyor. Ağırlıkları indirip **kendi bilgisayarında** çalıştırıyorsun — kredi yok, kuyruk yok, kare başına ücret yok. Bu kit kurulumu, ilk videoyu ve kodlama ajanına sahneleri kurgulatma akışını adım adım veriyor.

İÇİNDEKİLER

01 Model ne yapıyor, ne yapmıyor s.2
02 Lisans ve donanım gerçeği s.2
03 Kurulum — repo, bağımlılık, ağırlıklar s.3
04 İlk video: yazıdan videoya s.4
05 Görselden videoya ve bellek ayarları s.4
06 Sahneleri kodlama ajanına kurgulatmak s.5
07 Prompt yazma kuralları s.6
08 Başlangıç checklist'i s.6

01 · MODEL NE YAPIYOR

Tek model, *altı ayrı iş.*

LTX-2, video ve sesi ayrı ayrı üretip birleştiren bir zincir değil; ikisini **aynı geçişte** üreten tek bir DiT modeli. Depodaki hazır pipeline'lar şunları veriyor:

Yazıdan video

Prompt yaz, senkron sesli klip çıkar. En hızlı yol: **DistilledPipeline**.

`ltx_pipelines.distilled`

Görselden video

Elindeki kareyi başlangıç yapıp hareketi modele kurdurursun.

`--image ile`

Sesten video

Var olan bir ses dosyasına görüntü ürettirirsin.

`ltx_pipelines.a2vid_two_stage`

Retake

Bitmiş videonun **yalnızca bir zaman aralığını** yeniden üretir.

`ltx_pipelines.retake`

Keyframe arası

İki anahtar kare verirsin, aradaki hareketi model doldurur.

`ltx_pipelines.keyframe_interpolation`

Yalnız ses

Görüntüsüz, sadece ses üretimi de ayrı bir pipeline.

`ltx_pipelines.t2a_one_stage`

02 · LİSANS VE DONANIM

Açık ağırlık, *ama şartlı.*

LİSANSI BÖYLE OKU

Depo "LTX-2.x Community License" ile geliyor — MIT ya da Apache değil. Özet: **yıllık geliri 10 milyon doların altındaki** kişi ve şirketler serbestçe kullanabiliyor. Bu eşiğin üstündeki şirketlerin ticari kullanım için Lightricks'ten **ayrı ücretli lisans** alması gerekiyor; onlar için ücretsiz kullanım yalnızca araştırma, test ve deneme ile sınırlı. Tek kişi ve küçük ekip için pratikte engel yok — ama sözleşmeyi bir kez okumadan büyük bir müşteri işine sokma.

DONANIM BEKLENTİSİ

Bu 22 milyar parametrelik bir model — "her bilgisayarda döner" demek yanlış olur. Ağırlıklar onlarca GB yer kaplar ve tam boyutta çalışmak ciddi VRAM ister. Belleğin yetmediğinde model çalışmaz değil, **yavaşlar**: *--quantization fp8-cast* ile FP8'e düşürüp *--offload cpu* (ya da *disk*) ile ağırlıkları taşırsın. Kısa ve düşük çözünürlüklü bir klip ile başla, sonra büyüt.

03 · KURULUM

Üç adım: *repo, bağımlılık, ağırlık.*

Resmî depo **github.com/Lightricks/LTX-2**. Ağırlıklar Hugging Face'te ayrı duruyor ve **kapılı (gated)** — indirmeden önce model sayfasındaki şartları kabul edip giriş yapman gerekiyor.

ADIM 1 — DEPOYU KUR

terminal

```
$ git clone https://github.com/Lightricks/LTX-2.git
$ cd LTX-2
$ uv sync --extra natten
# natten yalnızca Linux + CUDA'da kurulur; Windows ve macOS'ta
# otomatik atlanır, çözümleme yavaş yola düşer — komut aynı kalır.
```

ADIM 2 — AĞIRLIKLARI İNDİR

terminal

```
$ hf auth login
# LTX-2.3 tek dosya: transformer + VAE'ler + text projection bir arada
$ hf download Lightricks/LTX-2.3 \
    ltx-2.3-22b-distilled-1.1.safetensors \
    ltx-2.3-spatial-upscaler-x2-1.1.safetensors \
    --local-dir models/ltx-2.3
# Metin kodlayıcı ayrı indirilir (LTX-2.3 için Gemma 3)
$ hf download google/gemma-3-12b-it-qat-q4_0-unquantized \
    --local-dir models/gemma-3-12b
```

401 / 403 ALIRSAN

Hata anahtarın bozuk demek değil: model kapılı. Hugging Face'te **LTX-2.3 sayfasındaki şartları kabul et** ve **Read** yetkili bir token ile giriş yap. İnce ayarlı (fine-grained) token kullanıyorsan "read gated repos" iznini açman gerekiyor.

SÜRÜM KARIŞIKLIĞINA DİKKAT

Depo artık **LTX-2.5**'i varsayılan öneriyor; 2.5 ağırlıkları **bileşen başına ayrı dosya** halinde geliyor ve 2.3 dosyalarıyla **karışmaz**. Bir LoRA yalnızca eğitildiği sürümle çalışır. Bu kit 2.3'ün tek-dosya (monolith) düzenini anlatıyor; 2.5'e geçersen bayrak isimleri *--transformer-path*, *--text-encoder-path* gibi ayrışır.

04 · İLK VİDEO

Yazıdan videoya, *tek komut.*

Damıtılmış (distilled) pipeline en hızlısı: 8 adımlık sabit bir program kullanıyor ve ayrıca yönlendirme (guidance) istemiyor. Başlangıç için önerilen yol bu.

terminal — yazıdan videoya

```
uv run python -m ltx_pipelines.distilled \
    --distilled-checkpoint-path models/ltx-2.3/ltx-2.3-22b-distilled-1.1.safetensors \
    --spatial-upsampler-path    models/ltx-2.3/ltx-2.3-spatial-upscaler-x2-1.1.safetensors \
    --gemma-root                models/gemma-3-12b \
    --num-frames 121 \
    --seed 42 \
    --output-path cikti.mp4 \
    --prompt "Yağmurdan sonra ıslak asfaltta yürüyen orta yaşlı bir adam,
      omuz hizasından takip eden kamera, neon tabelaların yansımaları
      zeminde titriyor, sığ alan derinliği, soğuk mavi tonlar, sabit ritim"
```

Kare sayısını *--num-frames*, çözünürlüğü *--height* ve *--width* belirler. *--seed* aynı kaldığı sürece aynı prompt aynı sonucu verir — bir sahneyi **kilitleyip** sadece tek bir detayı değiştirmek için bunu kullan.

05 · GÖRSELDEN VİDEOYA + BELLEK

Elindeki kareyi *harekete geçir.*

terminal — görselden videoya, düşük bellek

```
uv run python -m ltx_pipelines.distilled \
    --distilled-checkpoint-path models/ltx-2.3/ltx-2.3-22b-distilled-1.1.safetensors \
    --spatial-upsampler-path    models/ltx-2.3/ltx-2.3-spatial-upscaler-x2-1.1.safetensors \
    --gemma-root                models/gemma-3-12b \
    --image "kareler/urun.png" \
    --num-frames 121 --seed 42 --output-path urun_klip.mp4 \
    --quantization fp8-cast --offload cpu \
    --prompt "Ürün yavaşça kendi ekseninde dönüyor, yumuşak stüdyo ışığı"

# Hangi bayraklar var, tam liste:
uv run python -m ltx_pipelines.distilled --help
```

UZUN VİDEO NASIL YAPILIR

Model tek seferde kısa klip üretir; uzun videoyu **sahne sahne** üretip birleştirerek elde edersin. Devamlılığı korumanın en ucuz yolu: bir sahnenin son karesini bir sonraki çağrıya *--image* olarak vermek. Bu yüzden sıradaki bölüm önemli — sahneleri planlayıp bu komutları arka arkaya çalıştıran şey senin kodlama ajanın olacak.

06 · AJANLA ÇALIŞMA

Sahneleri *ajan kurgulasın.*

LTX-2 bir komut satırı aracı. Bu yüzden dosya okuyup komut çalıştırabilen **her** kodlama ajanı onu sürebilir — Claude Code, Codex, Antigravity, Cursor fark etmez. Senin işin fikri vermek; sahne listesini, prompt'ları ve komut zincirini ajan kurar.

ADIM 1
Brief
Konu, süre, ton, en-boy oranı

→

ADIM 2
Sahne planı
Ajan sahneleri ve prompt'ları yazar

→

ADIM 3
Üretim
Her sahne için CLI çağrısı

→

ADIM 4
Birleştirme
ffmpeg ile tek dosya

AJANA VERECEĞİN BRİEF — KOPYALA YAPIŞTIR

ajan promptu

```
# Rol: LTX-2.3 ile sahne sahne video üreten yapımcı
Depo kökündesin (Lightricks/LTX-2). Ağırlıklar models/ altında.

## Görev
Konu: [konunu buraya yaz]
Toplam süre: [örn. 30 saniye] · Ton: [örn. sinematik, sakin]
En-boy: [16:9 veya 9:16]

## Nasıl çalış
1. Önce plan.md yaz: her sahne için süre, kamera, ışık ve
   tek paragraflık görsel prompt. Onaymı al, üretime geçme.
2. Onayladıktan sonra her sahne için ltx_pipelines.distilled çağır;
   çıktıları sahneler/01.mp4, 02.mp4 ... olarak kaydet.
3. Devamlılık için bir sahnenin son karesini çıkar ve bir sonraki
   çağrıya --image olarak ver.
4. --seed değerlerini plan.md'ye yaz — beğenmediğim sahneyi
   aynı seed ile tek tek yeniden üretebilmeliyim.
5. Bitince ffmpeg ile birleştir, tek dosya ver.

## Kurallar
- Bayrakları uydurma: emin değilsen --help çıktısını oku.
- Her sahneyi üretmeden önce tek sahnede dene, ben onaylayınca devam et.
- Bellek hatasında --quantization fp8-cast --offload cpu ekle,
  çözünürlüğü düşür; sessizce sahne atlama.
```

NEDEN ÖNCE PLAN

Ajanı doğrudan üretime salarsan 20 dakikayı yanlış sahnelere harcarsın. Plan dosyası ucuz, üretim pahalı: **metni onayla, sonra kareyi üret.** Seed'leri plana yazdırman da aynı sebeple — beğenmediğin tek sahneyi baştan üretirsin, videonun tamamını değil.

@burhankocabiyik

07 · PROMPT KURALLARI

Görüntü yönetmeni gibi *yaz.*

Lightricks'in kendi rehberi net: **tek akan paragraf**, kronolojik sıra, 200 kelimeyi geçme. Madde madde sıfat yığmak değil, çekimi tarif etmek işe yarıyor.

1. **Ana eylemle başla**
   İlk cümle ne olduğunu söylesin — sahneyi kurmadan önce hareketi ver.

2. **Hareketi ve görünüşü netleştir**
   Jestler, yön, hız; karakter ve nesnelerin görünümü kesin tariflerle.

3. **Arka plan, kamera, ışık**
   Ortamı, kamera açısı ve hareketini, ışık ile renkleri açıkça yaz.

4. **Değişimi belirt**
   Klip içinde bir şey değişiyorsa (ani olay, ışık dönüşü) sona ekle.

SES DE PROMPT'TAN GELİYOR

Model sesi ayrı üretmiyor — konuşmayı, tonu ve ortam sesini **aynı prompt'un içinde** tarif edersin. Replik istiyorsan cümleyi tırnak içinde ver ve sesin tonunu yaz. Prompt'u yazmakta zorlanırsan *--enhance-prompt* bayrağı modelin kendi zenginleştiricisini devreye sokar.

08 · BAŞLANGIÇ

Bugün *ilk klibini* çıkar

- [ ] **1. Depoyu kur**
  git clone + uv sync — ağırlıkları indirmeden önce kurulumun sorunsuz bittiğini gör.

- [ ] **2. Şartları kabul et, ağırlıkları indir**
  Hugging Face'te LTX-2.3 sayfasını onayla, Read token ile gir, distilled checkpoint + upscaler + Gemma 3'ü çek.

- [ ] **3. Tek kısa klip üret**
  Düşük çözünürlük, az kare, sabit seed. Amaç güzel video değil — zincirin çalıştığını kanıtlamak.

- [ ] **4. Bellek bayraklarını ayarla**
  Yavaşlık ya da bellek hatası varsa fp8-cast + offload ile kendi makinenin sınırını bul, not et.

- [ ] **5. Ajanı devreye al**
  Brief prompt'unu ver, plan.md'yi onayla, sahneleri ürettir ve ffmpeg ile birleştir.

HATIRLATMA

Bu modelin asıl kazancı çözünürlük değil, **sınırsız deneme**. Bulut kredisi harcamadığın için "acaba şu açı daha mı iyi" sorusunu on kere sorabilirsin. İyi video, iyi tek prompt'tan değil; **ucuz tekrardan** çıkıyor.